

Continuous exposures and g-computation

Malcolm Barrett

Stanford University

Normal regression estimates associations. But we want *causal* estimates: what would happen if *everyone* in the study were exposed to x vs if *no one* was exposed.

G-Computation/G-Formula

- 1 Fit a model for $y \sim x + z$ where z is all covariates
- 2 Create a duplicate of your data set for each level of x
- 3 Set the value of x to a single value for each cloned data set (e.g $x = 1$ for one, $x = 0$ for the other)

G-Computation/G-Formula

- 1 Make predictions using the model on the cloned data sets
- 2 Calculate the estimate you want, e.g. $\text{mean}(x_1) - \text{mean}(x_0)$

Advantages of the parametric G-formula

Often more statistically precise than propensity-based methods

Incredibly flexible

Basis of other important causal models, e.g. causal survival analysis and TMLE

Greek Pantheon data (greek_data)

The name of a Greek god	A prognostic factor	The treatment, a heart transplant	The outcome, death
Rheia	0	0	0
Kronos	0	0	1
Demeter	0	0	0
Hades	0	0	0
Hestia	0	1	0
Poseidon	0	1	0
Hera	0	1	0
Zeus	0	1	1
Artemis	1	0	1
Apollo	1	0	1

+ 10 more rows

1. Fit a model for $y \sim a + 1$

```
1 greek_model <- lm(y ~ a + 1, data = greek_data)
```

2. Create a duplicate of your data set for each level of **a**

The name of a Greek god	A prognostic factor	The treatment, a heart transplant	The outcome, death
Rheia	0	0	0
Kronos	0	0	1
Demeter	0	0	0
Hades	0	0	0
Hestia	0	1	0
Poseidon	0	1	0
Hera	0	1	0
Zeus	0	1	1
Artemis	1	0	1
Apollo	1	0	1

2. Create a duplicate of your data set for each level of **a**

The name of a Greek god	A prognostic factor	The treatment, a heart transplant	The outcome, death
Rheia	0	0	0
Kronos	0	0	1
Demeter	0	0	0
Hades	0	0	0
Hestia	0	1	0
Poseidon	0	1	0
Hera	0	1	0
Zeus	0	1	1
Artemis	1	0	1
Apollo	1	0	1

The name of a Greek god	A prognostic factor	The treatment, a heart transplant	The outcome, death
Rheia	0	0	0
Kronos	0	0	1
Demeter	0	0	0
Hades	0	0	0
Hestia	0	1	0
Poseidon	0	1	0
Hera	0	1	0
Zeus	0	1	1
Artemis	1	0	1
Apollo	1	0	1

3. Set the value of **a** to a single value for each cloned data set

The name of a Greek god	A prognostic factor	a	The outcome, death
Rheia	0	0	0
Kronos	0	0	1
Demeter	0	0	0
Hades	0	0	0
Hestia	0	0	0
Poseidon	0	0	0
Hera	0	0	0
Zeus	0	0	1
Artemis	1	0	1
Apollo	1	0	1

The name of a Greek god	A prognostic factor	a	The outcome, death
Rheia	0	1	0
Kronos	0	1	1
Demeter	0	1	0
Hades	0	1	0
Hestia	0	1	0
Poseidon	0	1	0
Hera	0	1	0
Zeus	0	1	1
Artemis	1	1	1
Apollo	1	1	1

3. Set the value of **a** to a single value for each cloned data set

```
1 # set all participants to have a = 0
2 untreated_data <- greek_data |>
3   mutate(a = 0)
4
5 # set all participants to have a = 1
6 treated_data <- greek_data |>
7   mutate(a = 1)
```

4. Make predictions using the model on the cloned data sets

```
1 # predict under the data where everyone is untreated
2 predicted_untreated <- greek_model |>
3   augment(newdata = untreated_data) |>
4   select(untreated = .fitted)
5
6 # predict under the data where everyone is treated
7 predicted_treated <- greek_model |>
8   augment(newdata = treated_data) |>
9   select(treated = .fitted)
10
11 predictions <- bind_cols(
12   predicted_untreated,
13   predicted_treated
14 )
```

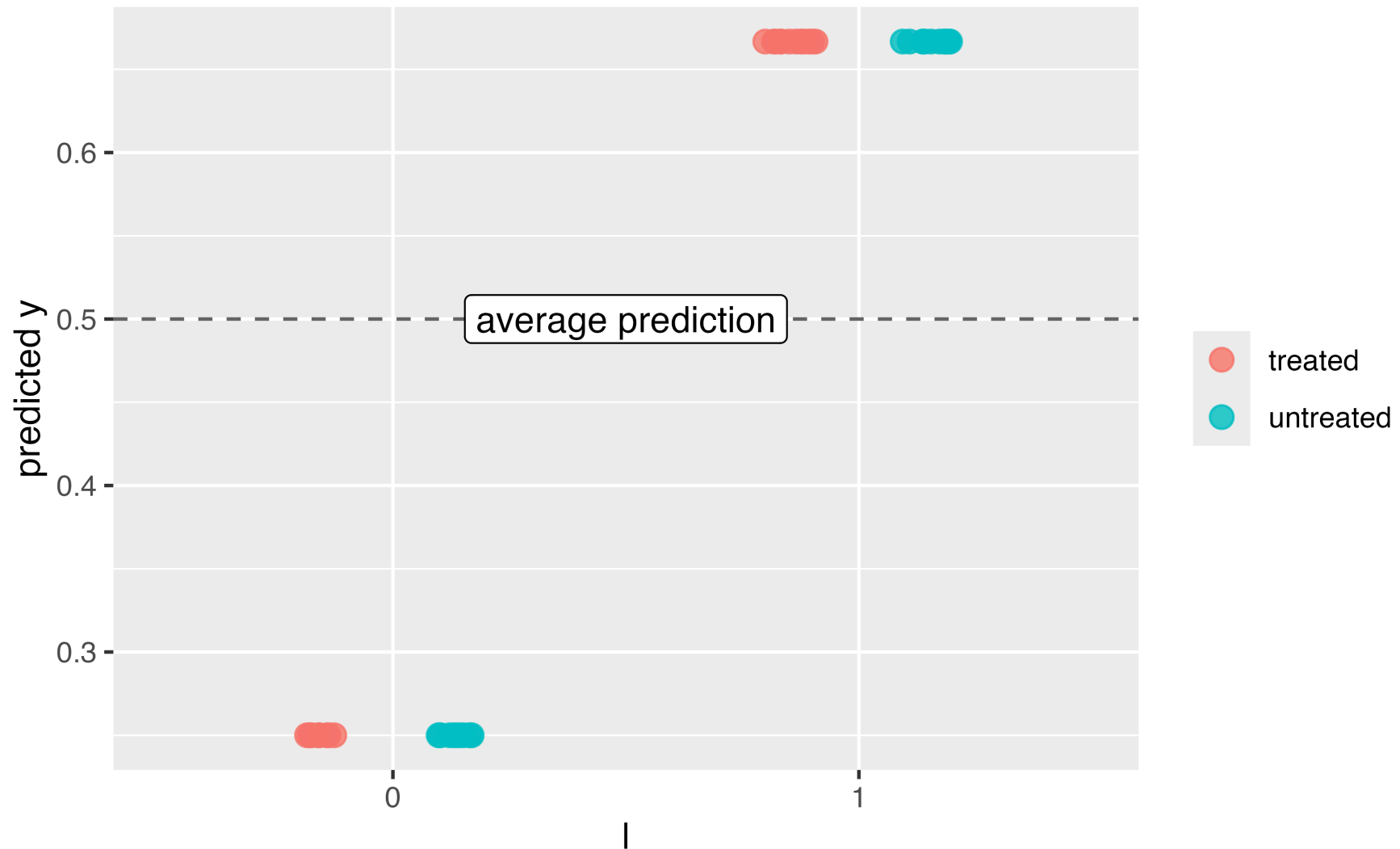
5. Calculate the estimate you want

```
1 predictions |>
2   summarise(
3     mean_treated = mean(treated),
4     mean_untreated = mean(untreated),
5     difference = mean_treated - mean_untreated
6   )
```

A tibble: 1 × 3

	mean_treated	mean_untreated	difference
	<dbl>	<dbl>	<dbl>
1	0.5	0.5	0

Marginalizing over the covariates



Predict at every observation's actual covariates, then average

avg_comparisons() does all five steps

```
1 library(marginaleffects)
2
3 avg_comparisons(greek_model, variables = "a")
```

```
# A tibble: 1 × 4
  estimate std.error conf.low conf.high
  <dbl>      <dbl>   <dbl>   <dbl>
1      0      0.24   -0.47    0.47
```

The standard error comes from the **delta method**, which approximates the variance of a transformation of the estimated coefficients

Targeting estimands with **newdata**

newdata changes whose covariates we average over

```
1  # ATT: average over the treated
2  avg_comparisons(
3    greek_model,
4    variables = "a",
5    newdata = filter(greek_data, a == 1)
6  )
7
8  # ATC: average over the untreated
9  avg_comparisons(
10   greek_model,
11   variables = "a",
12   newdata = filter(greek_data, a == 0)
13 )
```

Same model, three populations

```
# A tibble: 3 × 4
  estimand estimate conf.low conf.high
<chr>      <dbl>    <dbl>    <dbl>
1 ATE         0     -0.47     0.47
2 ATT         0     -0.47     0.47
3 ATC         0     -0.47     0.47
```

Without an exposure-covariate interaction, the three coincide: the model assumes a constant effect across covariate values

Tilting toward a target population

The ATO and ATM populations are not subsets of the rows, so weight the average by the tilting function instead

```
1 library(propensity)
2
3 greek_ps <- glm(a ~ 1, data = greek_data, family = binomial()) |>
4   predict(type = "response")
5
6 avg_comparisons(
7   greek_model,
8   variables = "a",
9   wts = ps_tilt(greek_ps, "ato")
10 )
```

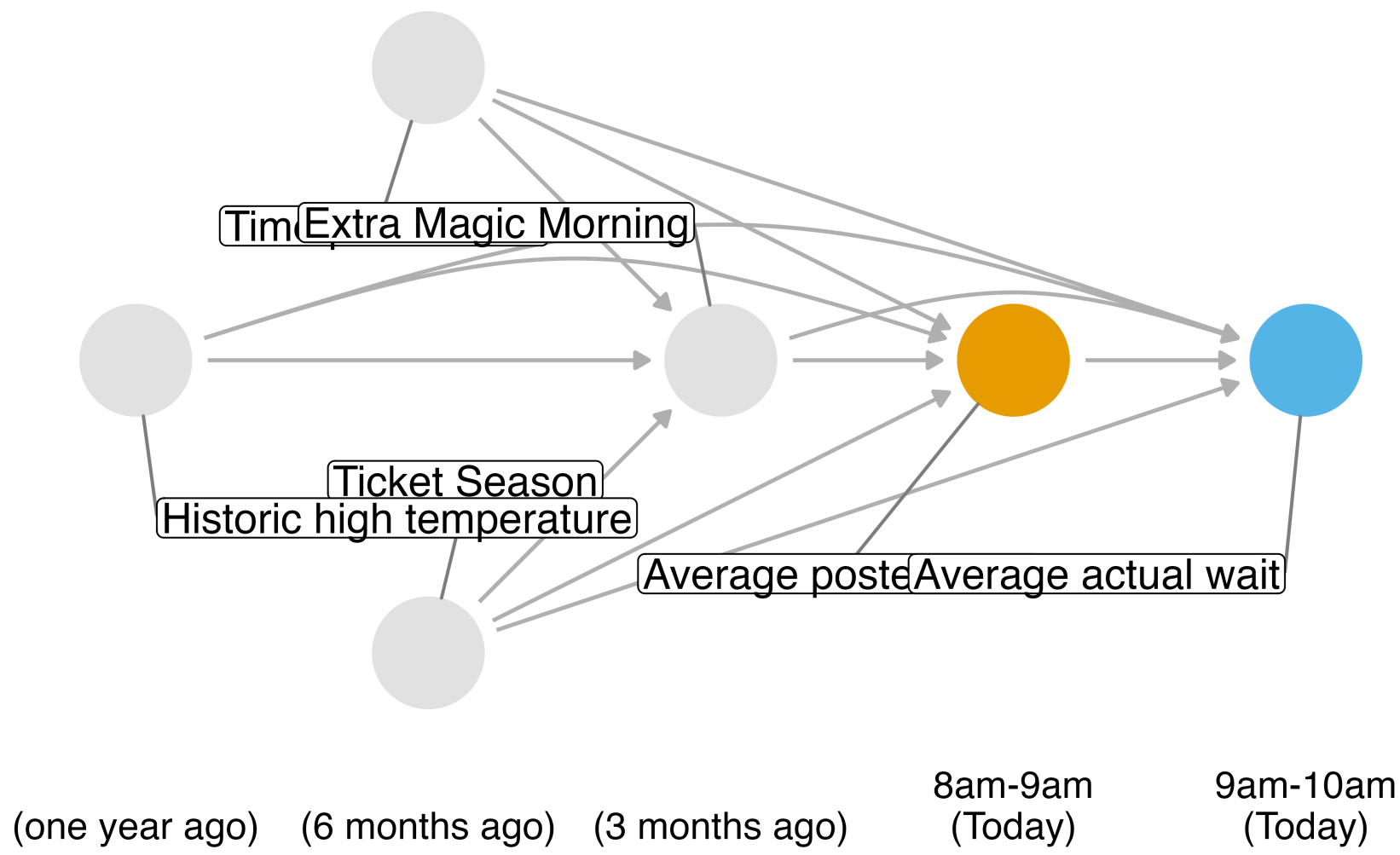
Continuous exposures

**We recommend g-computation
over propensity scores for
continuous exposures because of
stability issues**

When overlap is poor, IPW makes its reliance on overlap explicit: the weights blow up and we are forced to confront the problem.

G-computation absorbs the problem silently, producing a stable estimate that may nonetheless be wrong.

Do *posted* wait times at 8 am affect *actual* wait times at 9 am?



Your Turn

Work through Your Turns 1-4 in `11-continuous-g-computation-exercises.qmd`